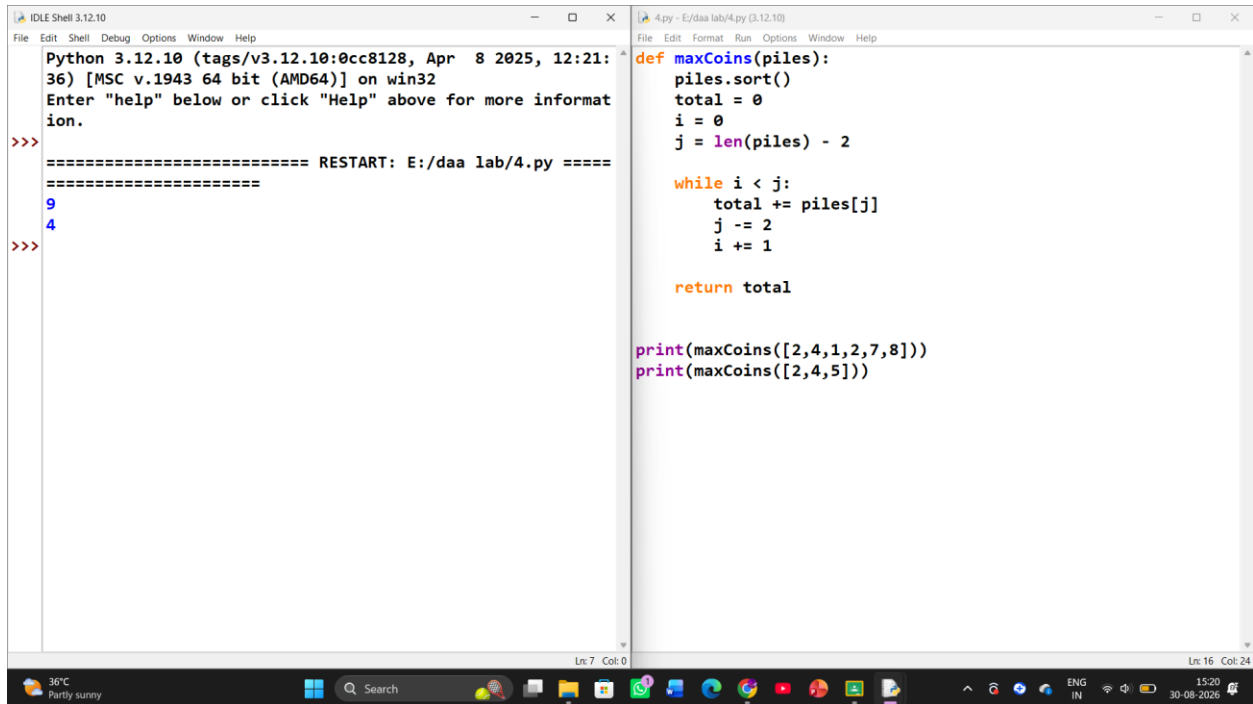


EXPERIMENT 1



The screenshot shows two windows in the Python IDLE environment. The left window is the 'Python Shell' (IDLE Shell 3.12.10) displaying the Python version (3.12.10), date (Apr 8 2025), and architecture (MSC v.1943 64 bit (AMD64) on win32). It shows a restart command and the execution of the 'maxCoins' function with two inputs: [2,4,1,2,7,8] and [2,4,5]. The right window is the '4.py' editor (E:/daa lab/4.py) containing the 'maxCoins' function definition. The function sorts the 'piles' list and uses a while loop to calculate the maximum number of coins by comparing adjacent elements. The function returns the 'total' value, which is then printed for both input lists.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>> 9
>>> 4
>>>
```

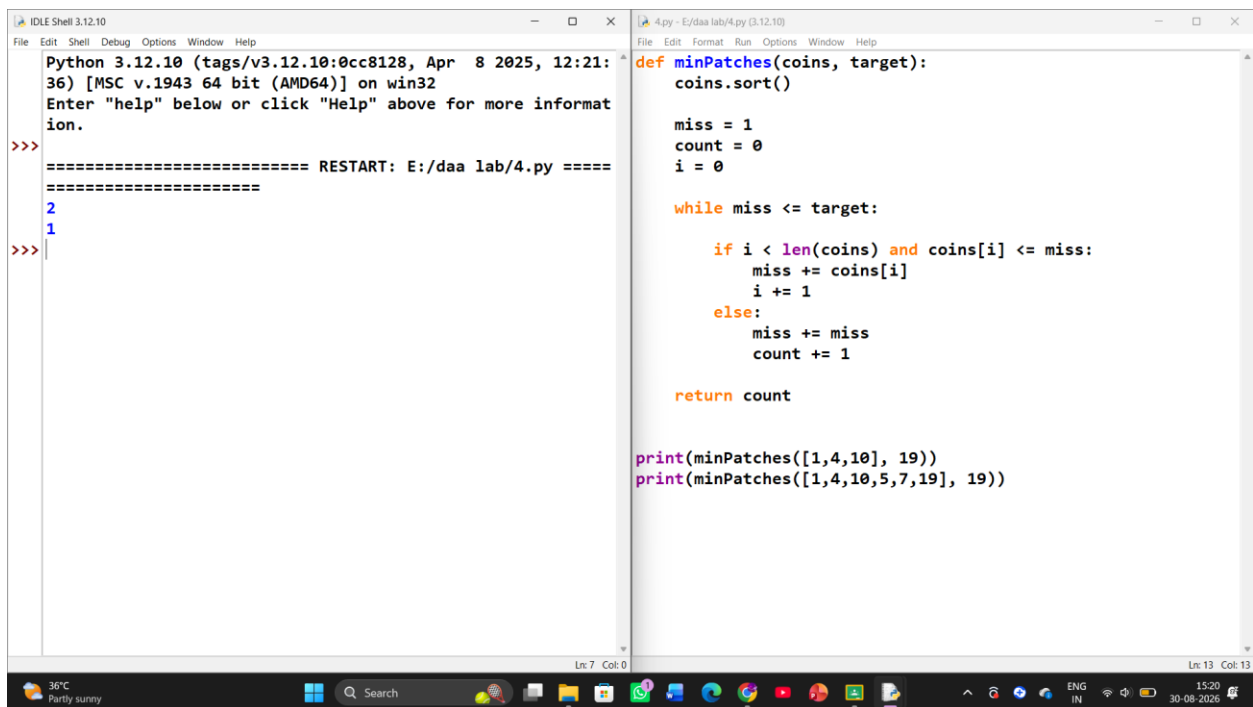
```
def maxCoins(piles):
    piles.sort()
    total = 0
    i = 0
    j = len(piles) - 2

    while i < j:
        total += piles[j]
        j -= 2
        i += 1

    return total

print(maxCoins([2,4,1,2,7,8]))
print(maxCoins([2,4,5]))
```

EXPERIMENT 2



The screenshot shows two windows in the Python IDLE environment. The left window is the 'Python Shell' (IDLE Shell 3.12.10) displaying the Python version (3.12.10), date (Apr 8 2025), and architecture (MSC v.1943 64 bit (AMD64) on win32). It shows a restart command and the execution of the 'minPatches' function with two inputs: [1,4,10] and [1,4,10,5,7,19]. The right window is the '4.py' editor (E:/daa lab/4.py) containing the 'minPatches' function definition. The function sorts the 'coins' list and uses a while loop to calculate the minimum number of patches by comparing the current element with the 'miss' value. The function returns the 'count' value, which is then printed for both input lists.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>> 2
>>> 1
>>>
```

```
def minPatches(coins, target):
    coins.sort()

    miss = 1
    count = 0
    i = 0

    while miss <= target:
        if i < len(coins) and coins[i] <= miss:
            miss += coins[i]
            i += 1
        else:
            miss += miss
            count += 1

    return count

print(minPatches([1,4,10], 19))
print(minPatches([1,4,10,5,7,19], 19))
```

EXPERIMENT 3

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
>>> 3
>>> 11

nonlocal answer

if index == len(jobs):
    answer = min(answer, max(workers))
    return

job = jobs[index]

for i in range(k):

    if workers[i] + job >= answer:
        continue

    if i > 0 and workers[i] == workers[i - 1]:
        continue

    workers[i] += job
    backtrack(index + 1)
    workers[i] -= job

    if workers[i] == 0:
        break

backtrack(0)
return answer

print(minimumTime([3,2,3], 3))
print(minimumTime([1,2,4,7,8], 2))
```

EXPERIMENT 4

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
>>> 120
>>> 120

dp = [0] * (n + 1)
end_times = [job[1] for job in jobs]

for i in range(1, n + 1):

    start, end, p = jobs[i - 1]

    previous = bisect_right(end_times, start, 0, i - 1)

    take = p + dp[previous]
    skip = dp[i - 1]

    dp[i] = max(take, skip)

return dp[n]

print(jobScheduling(
    [1,2,3,3],
    [3,4,5,6],
    [50,10,40,70]
))

print(jobScheduling(
    [1,2,3,4,6],
    [3,5,10,6,9],
    [20,20,100,70,60]
))
```

EXPERIMENT 5

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>> [0, 7, 3, 9, 5]
>>> [0, 5, 8, 9]

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help
    if graph[u][v] != float('inf'):
        new_distance = distance[u] + graph[u][v]

        if new_distance < distance[v]:
            distance[v] = new_distance

    return distance

INF = float('inf')

graph1 = [
    [0,10,3,INF,INF],
    [INF,0,1,2,INF],
    [INF,4,0,8,2],
    [INF,INF,INF,0,7],
    [INF,INF,INF,9,0]
]

graph2 = [
    [0,5,INF,10],
    [INF,0,3,INF],
    [INF,INF,0,1],
    [INF,INF,INF,0]
]

print(dijkstra(graph1, 0))
print(dijkstra(graph2, 0))

Ln: 7 Col: 0
Ln: 55 Col: 26
```

EXPERIMENT 6

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>> 20
>>> 5
>>>

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help
    return distance[target]

edges1 = [
    (0,1,7),
    (0,2,9),
    (0,5,14),
    (1,2,10),
    (1,3,15),
    (2,3,11),
    (2,5,2),
    (3,4,6),
    (4,5,9)
]

edges2 = [
    (0,1,10),
    (0,4,3),
    (1,2,2),
    (1,4,4),
    (2,3,9),
    (3,2,7),
    (4,1,1),
    (4,2,8),
    (4,3,2)
]

print(dijkstra(6, edges1, 0, 4))
print(dijkstra(5, edges2, 0, 3))

Ln: 7 Col: 0
Ln: 64 Col: 32
```

EXPERIMENT 7

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>>
a 00
b 01
c 10
d 11
>>>

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help
(f1 + f2, c1 + c2, new_tree)
)
tree = heap[0][2]
codes = {}

def generate(tree, code):

    if isinstance(tree, str):
        codes[tree] = code
        return

    left, right = tree

    generate(left, code + "0")
    generate(right, code + "1")

    generate(tree, "")

    return codes

characters = ['a','b','c','d']
frequencies = [5,9,12,13]

codes = huffman_codes(characters, frequencies)

for char in characters:
    print(char, codes[char])
```

EXPERIMENT 8

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>>
aacda
fefbdaab
>>>

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help
current = ""

return result

codes1 = {
    'a': '110',
    'b': '10',
    'c': '0',
    'd': '111'
}

encoded1 = "1101100111110"
print(decode_huffman(codes1, encoded1))

codes2 = {
    'a': '0',
    'b': '101',
    'c': '100',
    'd': '111',
    'e': '1101',
    'f': '1100'
}

encoded2 = "110011011100101111001011"
print(decode_huffman(codes2, encoded2))
```

EXPERIMENT 9

The screenshot shows two windows from the IDLE 3.12.10 environment. The left window is the IDLE Shell, displaying the Python version (3.12.10), the date and time (Apr 8 2025, 12:21:36), and the system architecture (MSC v.1943 64 bit (AMD64) on win32). It also shows a restart command and some input/output. The right window is the IDLE Editor, showing a Python script named '4.py' located at 'E:/daa lab/4.py'. The script defines a function 'maximum_weight(weights, capacity)' that sorts the weights in descending order and iterates through them to find the maximum total weight that can be carried without exceeding the capacity. The function returns the total weight. The script also includes two print statements to test the function with different weight lists and capacities.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
60
50
>>>
```

```
def maximum_weight(weights, capacity):
    weights.sort(reverse=True)
    total = 0
    for weight in weights:
        if total + weight <= capacity:
            total += weight
    return total

print(maximum_weight([10,20,30,40,50], 60))
print(maximum_weight([5,10,15,20,25,30], 50))
```

EXPERIMENT 10

The screenshot shows two windows from the IDLE 3.12.10 environment. The left window is the IDLE Shell, displaying the Python version (3.12.10), the date and time (Apr 8 2025, 12:21:36), and the system architecture (MSC v.1943 64 bit (AMD64) on win32). It also shows a restart command and some input/output. The right window is the IDLE Editor, showing a Python script named '4.py' located at 'E:/daa lab/4.py'. The script defines a function 'minimum_containers(weights, capacity)' that iterates through the weights and counts the number of containers needed to hold them without exceeding the capacity. The function returns the number of containers. The script also includes two print statements to test the function with different weight lists and capacities.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
4
5
>>>
```

```
def minimum_containers(weights, capacity):
    containers = 1
    current = 0
    for weight in weights:
        if current + weight <= capacity:
            current += weight
        else:
            containers += 1
            current = weight
    return containers

print(minimum_containers([5,10,15,20,25,30,35], 50))
print(minimum_containers([10,20,30,40,50,60,70,80], 100))
```

EXPERIMENT 11

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19
Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]
Total weight of MST: 16
>>>

4.py - E:/daa lab/4.py (3.12.10)
edges1 = [
    (0,1,10),
    (0,2,6),
    (0,3,5),
    (1,3,15),
    (2,3,4)
]

edges2 = [
    (0,1,2),
    (0,3,6),
    (1,2,3),
    (1,3,8),
    (1,4,5),
    (2,4,7),
    (3,4,9)
]

mst, total = kruskal(4, edges1)

print("Edges in MST:", mst)
print("Total weight of MST:", total)

mst, total = kruskal(5, edges2)

print("Edges in MST:", mst)
print("Total weight of MST:", total)
```

EXPERIMENT 12

```
IDLE Shell 3.12.10
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
Is the given MST unique? True
Is the given MST unique? False
>>>

4.py - E:/daa lab/4.py (3.12.10)
mst1 = [
    (2,3,4),
    (0,3,5),
    (0,1,10)
]

print("Is the given MST unique?",
      is_unique_mst(4, edges1, mst1))

edges2 = [
    (0,1,1),
    (0,2,1),
    (1,3,2),
    (2,3,2),
    (3,4,3),
    (4,2,3)
]

mst2 = [
    (0,1,1),
    (0,2,1),
    (1,3,2),
    (3,4,3)
]

print("Is the given MST unique?",
      is_unique_mst(5, edges2, mst2))
```

